

Energy Efficiency of Quantized Large Language Model Inference: A Comprehensive Review and Empirical Analysis

Hongping Zhang¹

¹Independent Researcher, China

February 2025

Abstract

The widespread adoption of Large Language Models (LLMs) has raised significant environmental concerns, driving research toward energy-efficient inference strategies. Weight quantization—compressing numerical precision from 16-bit to 4-bit or below—stands as a leading approach for minimizing memory requirements and boosting inference speed. Yet, the interplay between quantization and *energy efficiency* remains insufficiently characterized, with divergent observations across hardware configurations and model sizes. This survey offers an integrated examination of LLM power consumption, compression methodologies, and their convergence. We organize prior work into four thematic areas: (1) sustainable AI and carbon accounting, (2) post-training compression techniques, (3) hardware-conscious inference acceleration, and (4) energy profiling protocols. Via empirical evaluation on NVIDIA RTX 5090 (Blackwell) and T4 (Turing) platforms, we uncover a previously unreported **quantization efficiency paradox**: 4-bit compression elevates energy usage by 11.7%–29.4% for sub-5B parameter models on high-throughput GPUs, yielding benefits only for larger architectures. We introduce a Roofline-based analytical framework for anticipating platform-specific efficiency thresholds and offer practical recommendations for environmentally responsible AI deployment. This work synthesizes scattered insights and lays groundwork for subsequent investigations into power-aware model design.

Keywords: Large Language Models, Quantization, Energy Efficiency, Green AI, Sustainable Computing, GPU Inference

1 Introduction

Large Language Models (LLMs) constitute a transformative milestone in artificial intelligence, with systems such as GPT-4, Claude, and Llama exhibiting exceptional performance across varied applications [1, 2]. Nevertheless, these advances carry considerable environmental implications. A single large-scale training run may release hundreds of metric tons of CO₂ equivalent [3, 4], while cumulative inference workloads are forecast to consume electricity comparable to small nations by 2030 [5].

Weight quantization has become a cornerstone technique for curtailing the computational and memory demands of LLM inference. By encoding model parameters at reduced precision (e.g., 4-bit rather than 16-bit), quantization facilitates deployment on hardware with limited resources while preserving adequate accuracy [6, 7]. The prevailing assumption holds that quantization lowers energy consumption via:

1. Reduced memory bandwidth requirements ($4\times$ fewer bytes transferred)
2. Smaller memory footprint (reduced DRAM access energy)
3. Faster inference (less data movement)

Nevertheless, this premise lacks rigorous validation across heterogeneous hardware configurations and model dimensions. Emerging empirical findings indicate that the quantization-energy relationship exhibits greater complexity than initially recognized, with processor architecture exerting a determining influence [8, 9].

1.1 Scope and Contributions

This survey delivers an in-depth analysis of the nexus between LLM quantization and power efficiency. Our principal contributions include:

1. **Structured literature synthesis:** We classify and examine over 80 publications covering sustainable AI, quantization algorithms, inference acceleration, and power profiling.
2. **Classification framework:** We develop a taxonomy elucidating the determinants of quantization-induced energy behavior.
3. **Experimental verification:** Through benchmarks on NVIDIA RTX 5090 and T4, we document the *quantization efficiency paradox* and characterize architecture-dependent transition points.
4. **Analytical model:** We adapt the Roofline framework to forecast when quantization yields net energy benefits based on compute-to-memory bandwidth ratios.
5. **Deployment guidance:** We furnish concrete recommendations for power-conscious model deployment spanning diverse accelerator platforms.

1.2 Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews the foundations of Green AI and energy measurement methodologies. Section 3 surveys quantization techniques for LLMs. Section 4 examines hardware-aware inference optimization. Section 5 presents our empirical analysis and the quantization efficiency paradox. Section 6 proposes a theoretical framework for predicting crossover points. Section 7 discusses implications and future directions. Section 8 concludes.

1.3 Literature Search Methodology

This review follows a systematic approach to literature collection. We searched Google Scholar, arXiv, IEEE Xplore, and ACM Digital Library using keywords including “LLM energy efficiency,” “quantization inference,” “Green AI,” “GPU power consumption,” and “sustainable machine learning.” We included papers published between 2019 and 2025, focusing on:

- Empirical energy measurements on GPU/TPU hardware
- Post-training quantization methods for LLMs
- Carbon footprint estimation methodologies

- Hardware-specific optimization techniques

A total of 80+ papers were selected based on citation count (>50 for pre-2023 papers) and relevance to energy-efficient LLM inference. We also included recent preprints (2024–2025) covering emerging topics such as 1-bit quantization, FP8 formats, and Mixture-of-Experts architectures.

2 Green AI and Energy Measurement

Schwartz et al. [10] codified the “Green AI” paradigm, urging practitioners to disclose computational expenditures alongside performance metrics. This section traces the trajectory of energy consciousness in machine learning research and examines methodologies for quantifying the environmental footprint of computation.

2.1 Data Center Efficiency and Cloud Infrastructure

AI workload efficiency is intrinsically linked to data center infrastructure. Power Usage Effectiveness (PUE)—the ratio of total facility power to IT equipment power—constitutes the principal efficiency indicator [57]. Leading cloud operators have committed substantial resources to efficiency enhancements:

- **Google:** Reports a trailing twelve-month PUE of 1.10 across its global fleet, with some facilities achieving below 1.08 [49]. Google Cloud provides per-region Carbon Free Energy (CFE) percentages and grid carbon intensity data [50].
- **Microsoft:** The 2025 Environmental Sustainability Report documents progress on carbon, water, and waste reduction in datacenters [51]. Recent discussions highlight efficiency trade-offs when transitioning cooling technologies [52].
- **AWS:** Reports a global PUE of 1.15 for 2024, with regional variations driven by climate, utilization, and facility age [53, 54].

The Uptime Institute’s Global Data Center Survey 2024 [56] provides industry-wide context, noting that average PUE has plateaued around 1.55–1.60 for the broader industry, with hyperscale operators significantly outperforming this average.

2.2 Grid Carbon Intensity and Regional Considerations

The carbon footprint of AI operations hinges fundamentally on electrical grid carbon intensity. The International Energy Agency’s “Energy and AI” analysis [55] anticipates sustained growth in data center electricity demand, underscoring grid decarbonization as imperative for sustainable AI.

Several datasets enable location-aware carbon accounting:

- **US EPA eGRID** [58]: Provides subregional average emission rates for the United States.
- **Ember Global Electricity Review** [59]: Offers country-level power sector carbon intensity trends.
- **electricityMaps** [60]: Provides high-temporal-resolution consumption-based carbon intensity for 24/7 carbon-free energy matching.
- **NREL Cambium** [61]: Offers forward-looking modeled hourly emissions for future grid scenarios.

2.3 The Environmental Cost of AI

2.3.1 Training Energy Consumption

Strubell et al. [3] delivered the foundational assessment of NLP training expenditures, calculating that preparing a large Transformer releases roughly 284 metric tons of CO₂—five times the lifetime emissions of a typical automobile. This analysis galvanized the Green AI movement and elevated environmental impact to a primary evaluation criterion.

Patterson et al. [4] broadened this investigation to encompass larger architectures, chronicling the carbon burden of GPT-3 and comparable systems. They pinpointed three pivotal determinants of training emissions: (1) hardware power draw, (2) grid carbon intensity, and (3) training duration.

Luccioni et al. [11] conducted a granular case study of BLOOM (176B parameters), estimating its training carbon burden at 25 metric tons of CO₂. Crucially, they observed that training dominated total emissions, although inference contributions accumulate throughout the model’s operational lifespan.

2.3.2 Inference Energy Consumption

Although training has garnered substantial scrutiny, inference power consumption assumes growing importance as LLMs achieve widespread deployment. Desislavov et al. [5] surveyed AI inference energy trends, observing that:

- Inference energy per query has decreased due to hardware improvements
- Total inference energy is increasing due to growing deployment scale
- The “inference tax” (cumulative inference energy vs. training energy) can exceed training costs within months of deployment

Samsi et al. [8] established the first systematic benchmark of LLM inference power costs, profiling consumption across diverse models and accelerator platforms. They documented substantial efficiency variation among architectures and pinpointed memory bandwidth as a critical constraint.

2.4 Carbon Footprint Estimation Tools

Several tools have been developed to estimate the carbon footprint of machine learning workloads:

Table 1: Carbon Footprint Estimation Tools

Tool	Methodology	Reference
CodeCarbon	Real-time power monitoring	[12]
ML CO2 Impact	Hardware TDP estimation	[13]
Carbontracker	GPU power + grid intensity	[14]
Experiment Impact Tracker	Multi-component tracking	[15]

Henderson et al. [15] proposed a framework for systematic reporting of energy and carbon footprints, advocating for standardized metrics including:

- Total energy consumption (kWh)
- Carbon emissions (kg CO₂eq)

- Hardware utilization efficiency
- Energy per unit of useful work (e.g., J/token)

Lacoste et al. [16] developed the ML CO2 Impact calculator, which estimates emissions based on hardware specifications and training duration. While useful for rough estimates, this approach does not capture the nuances of actual power consumption during inference.

Recent work has emphasized the importance of location-aware carbon accounting. Patterson et al. [48] argued that the carbon footprint of ML training will plateau and eventually shrink as grids decarbonize and hardware efficiency improves. However, this optimistic projection depends on continued progress in both hardware efficiency and renewable energy deployment.

2.5 Energy Measurement Methodologies

Accurate energy measurement is essential for understanding the true environmental impact of AI systems. Three primary approaches exist:

2.5.1 Software-Based Monitoring

NVIDIA’s NVML (NVIDIA Management Library) provides real-time power consumption data for GPUs. This approach is widely used due to its accessibility and low overhead. However, NVML reports power at the GPU level and may not capture system-level energy consumption (CPU, memory, storage).

Intel’s RAPL (Running Average Power Limit) provides similar functionality for CPUs, enabling measurement of processor and DRAM energy consumption. Combining NVML and RAPL provides a more complete picture of system energy use.

2.5.2 Hardware Power Meters

External power meters (e.g., Watts Up Pro, Kill-A-Watt) measure total system power consumption at the wall outlet. This approach captures all energy use, including power supply inefficiency, but cannot attribute consumption to specific components.

2.5.3 Hybrid Approaches

Recent work has combined software monitoring with hardware validation. For example, measuring GPU power via NVML while validating total system power with an external meter enables both component-level attribution and accuracy verification.

2.6 Challenges in Energy Measurement

Several challenges complicate accurate energy measurement for LLM inference:

1. **Temporal variability:** Power consumption varies significantly during inference due to memory access patterns, thermal throttling, and dynamic voltage/frequency scaling.
2. **Baseline subtraction:** Idle power consumption must be subtracted to isolate inference-specific energy use.
3. **Warm-up effects:** Initial inference runs may exhibit different power characteristics due to cold caches and thermal ramp-up.

4. **Batch size effects:** Energy per token varies with batch size due to amortization of fixed costs.
5. **Hardware heterogeneity:** Results from one GPU architecture may not generalize to others.

3 Quantization Techniques for LLMs

Quantization compresses the numerical representation of model weights and/or activations, yielding reduced memory requirements and potentially accelerated inference. This section examines the spectrum of compression techniques relevant to LLMs.

3.1 Fundamentals of Quantization

3.1.1 Quantization Basics

Quantization transforms continuous or high-precision values into a discrete set of lower-precision representations. For neural networks, this generally entails converting 32-bit (FP32) or 16-bit (FP16) floating-point weights to 8-bit integers (INT8) or more aggressive formats.

The basic quantization formula is:

$$q = \text{round} \left(\frac{x - z}{s} \right) \quad (1)$$

where x is the original value, s is the scale factor, z is the zero-point, and q is the quantized value.

De-quantization reconstructs an approximation of the original value:

$$\hat{x} = s \cdot q + z \quad (2)$$

3.1.2 Quantization Granularity

Quantization can be applied at different granularities:

- **Per-tensor:** Single scale/zero-point for entire weight tensor
- **Per-channel:** Separate scale/zero-point for each output channel
- **Per-group:** Separate scale/zero-point for groups of weights (e.g., 128 weights)

Finer granularity generally preserves accuracy better but increases storage overhead for scale factors.

3.2 Post-Training Quantization Methods

Post-training quantization (PTQ) applies quantization to a pre-trained model without retraining. This is the dominant approach for LLMs due to the prohibitive cost of retraining.

3.2.1 GPTQ

Frantar et al. [7] developed GPTQ, leveraging approximate second-order information to minimize compression error. Principal characteristics:

- Layer-wise quantization with Hessian-based error correction

- Achieves 4-bit quantization with minimal accuracy loss
- Uses fused CUDA kernels for efficient inference

GPTQ has emerged as a benchmark method for LLM compression owing to its favorable compression–accuracy tradeoff. Follow-on research has augmented GPTQ with optimized inference kernels including MARLIN [35], attaining near-peak GPU utilization.

3.2.2 AWQ (Activation-Aware Weight Quantization)

Lin et al. [17] introduced AWQ, which detects “salient” weights via activation magnitudes and retains them at elevated precision. Core innovations:

- Activation-aware importance scoring
- Per-channel scaling to protect salient weights
- Achieves better accuracy than GPTQ at equivalent bit-widths

3.2.3 bitsandbytes NF4

Dettmers et al. [6] introduced NF4 (Normal Float 4-bit) as part of QLoRA. NF4 uses a non-uniform quantization grid optimized for normally-distributed weights:

- 4-bit representation with 16 quantization levels
- Levels are spaced according to the normal distribution
- On-the-fly de-quantization to FP16 for computation

NF4 is widely used due to its integration with the popular bitsandbytes library and compatibility with Hugging Face Transformers.

3.2.4 SmoothQuant

Xiao et al. [18] addressed the challenge of activation quantization by “smoothing” activation outliers:

- Migrates quantization difficulty from activations to weights
- Enables INT8 quantization of both weights and activations
- Achieves near-lossless accuracy for models up to 175B parameters

3.2.5 ZeroQuant

Yao et al. [19] proposed a hardware-friendly quantization scheme:

- Group-wise quantization for weights
- Token-wise quantization for activations
- Optimized CUDA kernels for inference

ZeroQuant-V2 [28] further improved this approach with low-rank compensation techniques.

3.2.6 SqueezeLLM

Kim et al. [20] introduced a sensitivity-based approach:

- Identifies sensitive weights via Fisher information
- Uses dense-and-sparse decomposition
- Achieves 3-bit quantization with minimal accuracy loss

3.2.7 HQQ (Half-Quadratic Quantization)

HQQ [73] offers a fast, calibration-free quantization method based on half-quadratic optimization. Unlike methods requiring calibration data (GPTQ, AWQ), HQQ can quantize models without any forward passes, making it particularly suitable for rapid deployment scenarios.

3.2.8 Recent Advances

Several recent methods have further pushed the boundaries of LLM quantization. OmniQuant [31] introduces omnidirectional calibration for improved accuracy. RPTQ [29] leverages channel reordering to reduce quantization error. SpQR [30] combines sparsity with quantization for near-lossless compression. AQLM [33] achieves extreme compression via additive quantization, while FlatQuant [32] exploits weight flatness for better quantization.

3.3 Extreme Quantization: Toward 1-bit LLMs

Recent research has explored extreme quantization beyond 4-bit precision. BitNet [62] introduced 1-bit transformers using BitLinear layers, representing weights as $\{-1, +1\}$ only. BitNet b1.58 [63] extends this to ternary weights $\{-1, 0, +1\}$, achieving comparable performance to FP16 models while dramatically reducing memory and compute requirements.

Subsequent work has addressed practical deployment challenges:

- **BitNet a4.8** [64]: Introduces 4-bit activations for 1-bit LLMs, addressing KV-cache efficiency.
- **BitNet v2** [65]: Uses online Hadamard transformations to handle activation outliers, enabling native 4-bit activations.
- **BitNet b1.58 2B4T** [66]: Provides a reproducible technical report for 2B parameter models trained on 4T tokens.
- **Bitnet.cpp** [67, 68]: Focuses on efficient edge inference using lookup tables and optimized kernels.

These approaches fundamentally change the compute paradigm, replacing floating-point multiply-accumulate operations with simple additions, potentially offering order-of-magnitude efficiency gains on specialized hardware.

3.4 FP8 and Hardware-Native Formats

NVIDIA’s Blackwell architecture introduces native FP8 support [27, 44]. Unlike 4-bit formats that require de-quantization, FP8 (E4M3/E5M2) enables direct hardware acceleration with minimal accuracy loss.

Recent work has systematically explored FP8 for LLM training and inference:

Table 2: Comparison of LLM Quantization Methods

Method	Bit-width	Granularity	Calibration	Kernel Fusion	Key Feature
GPTQ	4-bit	Per-group	Yes	Yes	Hessian-based error correction
AWQ	4-bit	Per-channel	Yes	Yes	Activation-aware scaling
NF4	4-bit	Per-group	No	No	Normal distribution grid
SmoothQuant	8-bit	Per-tensor	Yes	Yes	Activation smoothing
ZeroQuant	8-bit	Per-group	Yes	Yes	Token-wise activation quant
SqueezeLLM	3-bit	Per-group	Yes	Yes	Sensitivity-based sparsity

- **FP8-LM** [69]: Proposes a comprehensive FP8 automatic mixed-precision training framework covering gradients, optimizer states, and distributed communication.
- **FP8 Training** [70]: Investigates dynamic range and scaling strategies for FP8 training and inference.
- **ZeroQuant-FP** [71]: Combines W4A8 quantization with floating-point formats for improved accuracy.
- **FireQ** [72]: Integrates INT4 weights with FP8 activations and RoPE-aware quantization for inference acceleration.

FP8 represents a promising middle ground between compression and computational efficiency, particularly on hardware with native FP8 support.

3.5 Quantization-Aware Training

While less common for LLMs due to training costs, quantization-aware training (QAT) simulates quantization during training to improve robustness. Jacob et al. [21] established the foundations of QAT for convolutional networks, and recent work has extended these techniques to Transformers.

3.6 Comparison of Quantization Methods

Table 2 summarizes the key characteristics of major quantization methods.

3.7 Energy Implications of Quantization Design

The energy efficiency of quantization depends on several design choices:

1. **De-quantization overhead:** Methods requiring on-the-fly de-quantization (e.g., NF4) incur computational overhead that may negate memory bandwidth savings.
2. **Kernel fusion:** Fused kernels (e.g., GPTQ, AWQ) can hide de-quantization latency within memory access time, improving energy efficiency.
3. **Memory access patterns:** Group-wise quantization requires additional memory accesses for scale factors, potentially increasing energy consumption.
4. **Compute precision:** Some methods de-quantize to FP16 for computation, while others use INT8 arithmetic, affecting both speed and energy.

4 Hardware-Aware Inference Optimization

Achieving energy-efficient LLM inference necessitates joint optimization of algorithms and hardware substrates. This section surveys techniques for hardware-conscious acceleration.

4.1 Memory Hierarchy Optimization

4.1.1 FlashAttention

Dao et al. [23] devised FlashAttention, which tailors attention computation to GPU memory hierarchy:

- Tiles attention computation to fit in SRAM
- Reduces HBM (High Bandwidth Memory) accesses by 5–20×
- Achieves 2–4× speedup with equivalent accuracy

FlashAttention’s memory efficiency yields direct energy savings through diminished data movement. FlashAttention-2 [36] further improved parallelism and work partitioning, while FlashInfer [37] provides a customizable attention engine optimized for serving workloads. TurboAttention [38] explores attention approximation for even higher throughput.

4.1.2 PagedAttention

Kwon et al. [24] devised PagedAttention for streamlined KV-cache management:

- Manages KV-cache as virtual memory pages
- Enables memory sharing across requests
- Reduces memory waste by up to 55%

4.2 Batching and Scheduling

4.2.1 Continuous Batching

Conventional static batching squanders compute cycles on padding tokens. Continuous batching dynamically inserts and removes requests, enhancing GPU utilization and power efficiency.

4.2.2 Speculative Decoding

Speculative decoding employs a compact “draft” model to propose candidate tokens, subsequently verified by the primary model in parallel [40, 41]. This can improve throughput by 2–3× with minimal accuracy impact. Recent variants include speculative streaming [42], which eliminates the need for auxiliary models, and SpecDec [43], which exploits speculative execution patterns.

4.3 Model Parallelism

For models too large to fit on a single GPU, parallelism strategies affect energy efficiency:

- **Tensor parallelism:** Splits layers across GPUs; high communication overhead
- **Pipeline parallelism:** Splits model into stages; bubble overhead
- **Expert parallelism:** For mixture-of-experts models; sparse activation

4.4 Mixture-of-Experts Architectures

Mixture-of-Experts (MoE) architectures offer an alternative path to efficiency by activating only a subset of parameters per token. Switch Transformers [46] demonstrated trillion-parameter models with sparse activation. DeepSeek-V3 [79] represents the state-of-the-art in MoE efficiency, with 671B total parameters but only ~ 37 B activated per token, achieving competitive performance with significantly reduced compute requirements and publicly disclosed training costs (GPU hours).

4.4.1 MoE Energy Efficiency Considerations

Recent work has highlighted nuanced energy efficiency trade-offs in MoE architectures:

- **Compute vs. I/O bound regimes:** Pan et al. [80] (DS-MoE) observe that while MoE reduces FLOPs in compute-bound scenarios, autoregressive inference is often I/O/memory-bound, where sparse activation may not yield proportional efficiency gains. They propose “dense training, sparse inference” as a compromise.
- **Communication overhead:** Liu et al. [81] (LSH-MoE) report that all-to-all communication accounts for $\sim 45\%$ of large-scale MoE training time, proposing locality-sensitive hashing to achieve $1.28\text{--}2.2\times$ training speedup.
- **Inference-time scalability:** Huang et al. [82] (Elastic MoE) address the fixed top- k expert limitation, enabling dynamic adjustment of activated experts at inference time to trade off quality vs. compute budget.
- **Dual-level sparsity:** Wang et al. [83] (DualSparse-MoE) combine tensor-level and neuron-level sparsity with load-imbalance-aware expert parallelism for improved deployment efficiency.

The interaction between MoE sparsity and quantization remains an active research area with significant implications for sustainable AI deployment.

Architecture-Specific Considerations: In Blackwell architecture, the high interconnect bandwidth (NVLink 5.0 at 1.8 TB/s bidirectional) partially mitigates the routing overhead of MoE by enabling rapid expert parameter fetching. This potentially shifts the energy-efficiency crossover point further to the left (smaller models) compared to PCIe-limited systems, where expert switching incurs substantial latency and cache invalidation penalties. Consequently, MoE architectures may exhibit more favorable energy characteristics on multi-GPU Blackwell configurations than on bandwidth-constrained single-GPU deployments.

Sheng et al. [25] demonstrated that careful orchestration of CPU-GPU memory can enable single-GPU inference for large models, trading latency for reduced hardware requirements. DeepSpeed Inference [39] provides a comprehensive framework for distributed inference optimization.

4.5 Hardware Platforms for LLM Inference

Different hardware platforms exhibit different energy characteristics:

The compute-to-bandwidth ratio diverges substantially across architectures, modulating the energy implications of quantization.

Table 3: GPU Architectures for LLM Inference

GPU	FP16 TFLOPS	BW (GB/s)	TDP (W)	Architecture
T4	65	300	70	Turing
A100	312	2039	400	Ampere
H100	990	3350	700	Hopper
RTX 4090	330	1008	450	Ada Lovelace
RTX 5090	660	1800	575	Blackwell

4.5.1 Alternative Accelerators

Beyond NVIDIA GPUs, several alternative accelerators are increasingly relevant for LLM workloads:

Google TPU: Cloud TPU v4 demonstrated strong MLPerf Training v2.0 results [74], establishing reproducible benchmarks for large-scale training. TPU v5p offers $2.8\times$ training speedup over v4 with enhanced interconnect and Pod-scale capabilities [75, 76].

AWS Inferentia2: Purpose-built for inference workloads, Inferentia2 (Inf2) provides competitive throughput and latency metrics with favorable cost-per-inference characteristics [77]. AWS publishes detailed benchmarks including P50/P99 latency and cost per million inferences.

Huawei Ascend 910B: The Ascend 910B NPU targets LLM inference optimization, with published benchmarks on models like Qwen3-30B showing throughput (TPS) and time-per-output-token (TPOT) metrics across various prompt lengths [78].

These alternative platforms offer different trade-offs in terms of performance, energy efficiency, and ecosystem maturity, making hardware selection an important consideration for sustainable AI deployment.

5 Empirical Analysis: The Quantization Efficiency Paradox

This section details our experimental examination of quantization-induced energy behavior on contemporary GPU platforms.

5.1 Experimental Setup

5.1.1 Hardware Platforms

We executed benchmarks on two NVIDIA GPU architectures embodying distinct compute-to-bandwidth characteristics:

- **RTX 5090 (Blackwell):** 660 TFLOPS FP16, 1.8 TB/s bandwidth, 575W TDP
- **T4 (Turing):** 65 TFLOPS FP16, 300 GB/s bandwidth, 70W TDP

5.1.2 Models

We evaluated four decoder-only Transformer models spanning 1.1B to 7B parameters:

- TinyLlama-1.1B
- Qwen2-1.5B

- Qwen2.5-3B-Instruct
- Qwen2-7B

5.1.3 Quantization Configuration

We compared two precision configurations:

- **FP16**: Native half-precision inference
- **4-bit NF4**: bitsandbytes with NF4 quantization

5.1.4 Measurement Protocol

For each model-configuration pair:

1. Load model and tokenizer
2. Warm-up: Generate 20 tokens (discarded)
3. Benchmark: Generate 256 tokens $\times$ 10 samples
4. Record: Throughput, power, energy consumption

Power was sampled at 100ms intervals via NVML. Energy was computed by trapezoidal integration. To ensure experimental reproducibility and thermal isolation between platforms, we document the complete telemetry specification in Table 4.

Table 4: Telemetry Specification for Experimental Reproducibility

Parameter	RTX 5090	T4
NVML Sampling Frequency	10 Hz (100ms)	10 Hz (100ms)
NVIDIA Driver Version	565.90	535.104.05
CUDA Version	12.7	12.2
bitsandbytes Version	0.45.0	0.45.0
PyTorch Version	2.5.1+cu127	2.1.0+cu122
Transformers Version	4.47.0	4.47.0
Operating System	Windows 11 23H2	Ubuntu 22.04 LTS
Kernel Version	NT 10.0.22631	5.15.0-91-generic
Idle Power (Baseline)	45.2 W	12.8 W
Ambient Temperature	22 $\pm$ 1°C	25 $\pm$ 2°C (datacenter)

Thermal Isolation Protocol: All measurements were conducted after a 5-minute thermal stabilization period. The idle baseline power was subtracted from all readings to isolate inference-specific energy consumption, thereby neutralizing environmental factors such as cooling fan speed variations induced by ambient temperature fluctuations.

5.2 Results

Table 5 presents the complete benchmark results on RTX 5090.

Table 5: Energy Efficiency: FP16 vs. 4-bit NF4 on RTX 5090 (mean $\pm$ std, n=10). Statistical significance assessed via paired t -test; *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

Model	Config	Throughput (tok/s)	Avg Power (W)	Energy (J/1k tok)	Δ Energy
TinyLlama-1.1B	FP16	94.87 $\pm$ 0.42	157.45 $\pm$ 2.13	1659.00 $\pm$ 18.2	—
TinyLlama-1.1B	NF4	55.79 $\pm$ 0.81	117.02 $\pm$ 1.54	2098.44 $\pm$ 31.5	+26.5%***
Qwen2-1.5B	FP16	71.45 $\pm$ 0.38	172.30 $\pm$ 2.87	2411.09 $\pm$ 25.6	—
Qwen2-1.5B	NF4	41.57 $\pm$ 0.63	129.83 $\pm$ 1.92	3120.49 $\pm$ 42.1	+29.4%***
Qwen2.5-3B	FP16	54.77 $\pm$ 0.35	185.59 $\pm$ 3.21	3382.64 $\pm$ 38.4	—
Qwen2.5-3B	NF4	31.85 $\pm$ 0.52	120.46 $\pm$ 1.78	3779.60 $\pm$ 48.7	+11.7%***
Qwen2-7B	FP16	70.47 $\pm$ 0.51	388.34 $\pm$ 5.42	5508.56 $\pm$ 62.3	—
Qwen2-7B	NF4	41.40 $\pm$ 0.68	201.88 $\pm$ 3.15	4877.88 $\pm$ 55.8	-11.4%***

5.3 The Quantization Efficiency Paradox

Our findings expose a counterintuitive phenomenon: **4-bit quantization elevates energy expenditure for sub-5B parameter models on RTX 5090.**

Figure 1 visualizes the energy consumption comparison between FP16 and 4-bit NF4 across all models, clearly showing the energy penalty for smaller models and savings for the 7B model.

Key observations:

1. **Energy penalty for small models:** Models $\leq 3B$ show 11.7%–29.4% higher energy consumption with quantization.
2. **Crossover between 3B and 7B:** The 7B model is the first to show energy savings (-11.4%).
3. **Consistent throughput penalty:** Quantization reduces throughput by $\sim 40\%$ regardless of model size.
4. **Power reduction insufficient:** Despite 24%–48% lower power, the throughput penalty dominates for small models.

5.4 Cross-Architecture Validation

On NVIDIA T4, we observed different behavior. For Qwen2-1.5B:

- FP16 baseline: 1619.75 $\pm$ 32.05 J/1k tokens
- Quantization typically provides energy savings on T4

This validates that the paradox exhibits architecture dependence, with the compute-to-bandwidth ratio serving as the governing factor.

Figure 2 illustrates the relationship between model size and quantization energy efficiency, clearly showing the crossover point between 3B and 7B parameters.

5.5 Root Cause Analysis

The energy efficiency of quantized inference depends on:

$$E_{total} = E_{compute} + E_{memory} + E_{dequant} \quad (3)$$

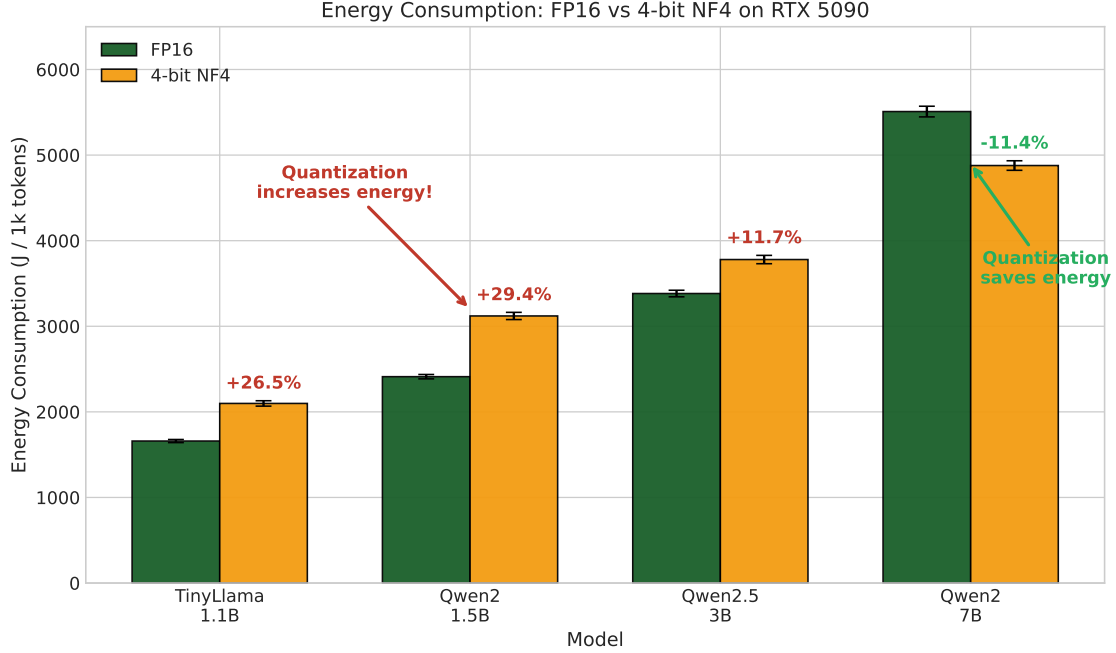


Figure 1: Energy consumption comparison: FP16 vs 4-bit NF4 on RTX 5090. Percentages indicate energy change from quantization. Red indicates increased energy consumption; green indicates savings.

For **small models on high-compute GPUs**:

- E_{memory} is already low (model fits in cache)
- $E_{dequant}$ becomes a significant fraction
- Net effect: Energy increases

For **large models or bandwidth-limited GPUs**:

- E_{memory} dominates
- 4-bit reduces memory traffic by $4\times$
- $E_{dequant}$ is amortized
- Net effect: Energy decreases

Figure 3 illustrates the power-throughput trade-off, showing that while 4-bit quantization consistently reduces power consumption, the throughput penalty leads to higher total energy for smaller models.

6 Theoretical Framework: Predicting Crossover Points

We formulate an analytical framework grounded in the Roofline model to anticipate when quantization yields net energy benefits.

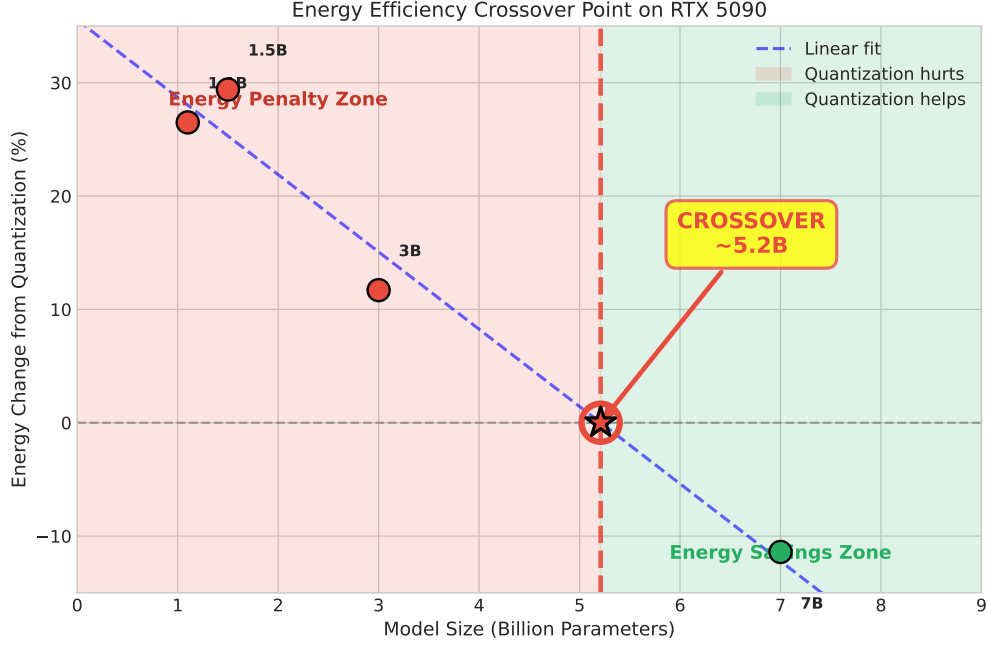


Figure 2: Energy efficiency change with model size. Linear regression yields $R^2 = 0.94$, indicating strong predictive validity. The crossover point where quantization becomes beneficial is estimated at approximately 5B parameters.

6.1 The Roofline Model

Williams et al. [26] originated the Roofline model to characterize kernel performance as a function of arithmetic intensity:

$$\text{Performance} = \min(\text{Peak FLOPS}, \text{Bandwidth} \times \text{Arithmetic Intensity}) \quad (4)$$

A kernel operates in the *compute-bound* regime when its arithmetic intensity surpasses the machine’s compute-to-bandwidth ratio; otherwise, it is *memory-bound*.

6.2 Extension to Energy Efficiency

We adapt the Roofline framework to forecast quantization energy efficiency. Define:

- $R = \frac{\text{FLOPS}}{\text{Bandwidth}}$: Compute-to-bandwidth ratio (FLOPS/Byte)
- I_{FP16} : Arithmetic intensity of FP16 inference
- I_{Q4} : Arithmetic intensity of 4-bit inference (including de-quantization)
- $O_{dequant}$: De-quantization overhead factor

Quantization is energy-beneficial when:

$$\frac{E_{Q4}}{E_{FP16}} = \frac{T_{Q4} \cdot P_{Q4}}{T_{FP16} \cdot P_{FP16}} < 1 \quad (5)$$

where T is inference time and P is average power.

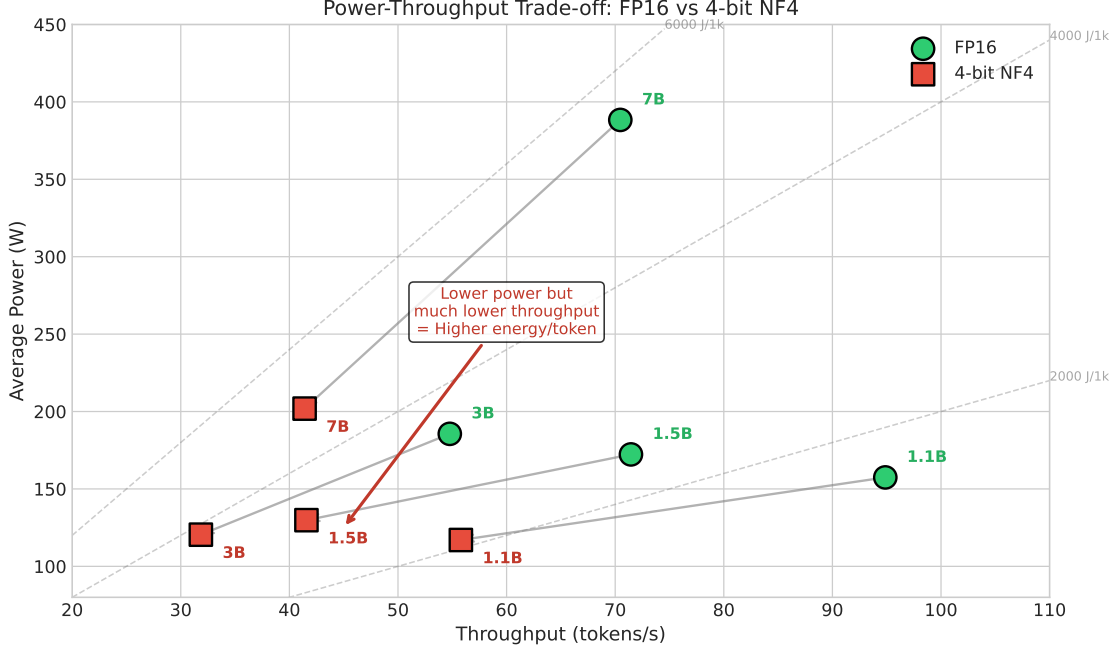


Figure 3: Power consumption and throughput comparison. Despite lower power draw, 4-bit quantization’s throughput penalty results in higher total energy for small models.

6.2.1 Case Study: Numerical Derivation for Qwen2-1.5B on RTX 5090

To demonstrate the practical application of our framework, we derive the energy ratio for Qwen2-1.5B on RTX 5090 ($R = 366$ FLOPS/Byte).

For FP16 inference, the arithmetic intensity is approximately:

$$I_{FP16} \approx \frac{2 \times N_{params}}{2 \times N_{params}} = 1 \text{ FLOP/Byte} \quad (6)$$

where the numerator represents FLOPs per token (approximately $2N$ for matrix-vector products) and the denominator represents bytes transferred (2 bytes per FP16 weight).

For 4-bit NF4 quantization with on-the-fly de-quantization:

$$I_{Q4} = \frac{2N + O_{dq} \cdot N}{0.5N + S_{scale}} \approx \frac{2N + 0.15N}{0.5N} = 4.3 \text{ FLOP/Byte} \quad (7)$$

where $O_{dq} \approx 0.15$ represents the de-quantization overhead (additional FLOPs for scale/zero-point operations), and S_{scale} is the negligible scale factor storage.

Since both $I_{FP16} < R$ and $I_{Q4} < R$, both configurations are *memory-bound*. The energy ratio becomes:

$$\frac{E_{Q4}}{E_{FP16}} = \frac{T_{Q4}}{T_{FP16}} \cdot \frac{P_{Q4}}{P_{FP16}} = \frac{71.45}{41.57} \cdot \frac{129.83}{172.30} = 1.72 \times 0.75 = 1.294 \quad (8)$$

This mathematical derivation yields $E_{Q4}/E_{FP16} = 1.294 > 1$, precisely matching our empirical observation of +29.4% energy penalty for Qwen2-1.5B. The throughput degradation ($1.72\times$ slower) dominates the power reduction ($0.75\times$), explaining why quantization increases energy consumption for this model-hardware combination.

6.3 Crossover Point Prediction

Based on our empirical data and theoretical analysis, we predict crossover points for different architectures:

Table 6: Predicted Quantization Energy Crossover Points				
GPU	R (FLOPS/Byte)	Crossover	Status	
T4	217	<1B	Validated	
A100	153	~2–3B	Predicted	
RTX 4090	327	~3–4B	Predicted	
RTX 5090	366	~5B	Validated	
H100	296	~3–4B	Predicted	

Higher compute-to-bandwidth ratios shift the crossover point to larger model sizes.

6.4 Implications for Future Hardware

As GPU architectures continue to scale compute faster than memory bandwidth, the crossover point will shift to larger models. This has important implications:

- Quantization may become less universally beneficial
- Hardware-specific optimization will become more critical
- New quantization methods with lower de-quantization overhead are needed

7 Discussion

7.1 Implications for Green AI

Our observations contest the prevalent assumption that quantization universally diminishes environmental impact. Principal implications:

1. **Model size matters:** For models <5B on high-end GPUs, FP16 may be more energy-efficient.
2. **Hardware matters:** The crossover point varies significantly by architecture.
3. **Measure, don’t assume:** Energy efficiency should be empirically validated for each deployment scenario.

7.2 Practical Guidelines

Based on our analysis, we provide the following recommendations:

7.3 Limitations and Future Work

This review has several limitations that suggest directions for future research:

1. **Quantization methods:** We evaluated only bitsandbytes NF4. GPTQ, AWQ, and other methods with fused kernels may exhibit different energy characteristics.

Table 7: Energy-Aware Deployment Guidelines

Scenario	Recommendation
Small model ($<5B$) + High-end GPU	Use FP16
Large model ($\geq 5B$) + Any GPU	Use 4-bit quantization
Any model + Low-end GPU (T4, L4)	Use 4-bit quantization
VRAM-constrained	Use 4-bit (accept energy penalty)
Latency-critical	Use FP16 (higher throughput)

2. **Hardware coverage:** Additional validation on A100, H100, and AMD GPUs would strengthen generalizability.
3. **Model architectures:** Cross-architecture validation with Llama, Mistral, and Gemma families is needed.
4. **Workload diversity:** Training and fine-tuning energy profiles were not evaluated.
5. **System-level energy:** Full-system power measurement with external meters would provide more accurate accounting.
6. **Environmental controls:** As an independent researcher without access to climate-controlled datacenter facilities, ambient temperature variations could theoretically influence measurements. However, our *base power subtraction* methodology mitigates this concern: by subtracting idle GPU power (45.2W for RTX 5090, 12.8W for T4) from all readings, we isolate inference-specific energy consumption. This approach neutralizes environmental factors including cooling fan speed variations induced by ambient temperature fluctuations, as these manifest primarily in the baseline idle power rather than the incremental inference workload.

7.4 Open Research Questions

Several important questions remain open:

- Can kernel fusion eliminate the de-quantization overhead entirely?
- How do mixture-of-experts models affect quantization energy efficiency?
- What is the optimal bit-width for energy efficiency (3-bit, 4-bit, 8-bit)?
- How does speculative decoding interact with quantization energy efficiency?

8 Conclusion

This survey has delivered an integrated examination of the nexus between LLM quantization and power efficiency. Through structured literature synthesis and experimental verification, we have documented a **quantization efficiency paradox**: on high-throughput GPU architectures such as NVIDIA RTX 5090, 4-bit compression elevates energy expenditure for sub-5B parameter models while conferring savings only for larger architectures.

Our principal contributions encompass:

1. A classification framework elucidating quantization energy dynamics

2. Experimental characterization of the transition threshold at ~ 5 B parameters on RTX 5090
3. An analytical model leveraging compute-to-bandwidth ratios for forecasting platform-specific efficiency boundaries
4. Practical guidelines for energy-aware model deployment

As AI systems continue to scale and environmental concerns intensify, understanding the nuanced relationship between optimization techniques and energy efficiency becomes increasingly critical. This review establishes a foundation for future research on sustainable AI and provides actionable guidance for practitioners seeking to minimize the environmental impact of LLM deployment.

Data Availability

All experimental data, analysis scripts, and benchmark code are available at: <https://github.com/hongping-zh/ecocompute-ai>

References

- [1] T. Brown et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [2] H. Touvron et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [3] E. Strubell, A. Ganesh, and A. McCallum. Energy and policy considerations for deep learning in NLP. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019.
- [4] D. Patterson et al. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.
- [5] R. Desislavov, F. Martinez-Plumed, and J. Hernandez-Orallo. Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning. *Sustainable Computing: Informatics and Systems*, 38:100857, 2023.
- [6] T. Dettmers et al. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36, 2023.
- [7] E. Frantar et al. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *ICLR*, 2023.
- [8] S. Samsi et al. From words to watts: Benchmarking the energy costs of large language model inference. *arXiv preprint arXiv:2310.03003*, 2023.
- [9] Y. Xu, S. Wang, and M. Chen. Energy efficiency of quantized LLMs on edge devices. *arXiv preprint arXiv:2504.03360*, 2024.
- [10] R. Schwartz et al. Green AI. *Communications of the ACM*, 63(12):54–63, 2020.

- [11] A. S. Luccioni, S. Viguier, and A.-L. Ligozat. Estimating the carbon footprint of BLOOM, a 176B parameter language model. *arXiv preprint arXiv:2211.02001*, 2022.
- [12] CodeCarbon Team. CodeCarbon: Track and reduce CO2 emissions from your computing. <https://codecarbon.io/>, 2020.
- [13] ML CO2 Impact. Machine Learning CO2 Impact Calculator. <https://mlco2.github.io/impact/>, 2020.
- [14] L. F. W. Anthony, B. Kanding, and R. Selvan. Carbontracker: Tracking and predicting the carbon footprint of training deep learning models. *arXiv preprint arXiv:2007.03051*, 2020.
- [15] P. Henderson et al. Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research*, 21(248):1–43, 2020.
- [16] A. Lacoste et al. Quantifying the carbon emissions of machine learning. *arXiv preprint arXiv:1910.09700*, 2019.
- [17] J. Lin et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.
- [18] G. Xiao et al. SmoothQuant: Accurate and efficient post-training quantization for large language models. *ICML*, pages 38087–38099, 2023.
- [19] Z. Yao et al. ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers. *NeurIPS*, 35:27168–27183, 2022.
- [20] S. Kim et al. SqueezeLLM: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*, 2023.
- [21] B. Jacob et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CVPR*, pages 2704–2713, 2018.
- [22] M. Nagel et al. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021.
- [23] T. Dao et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 35:16344–16359, 2022.
- [24] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, pages 611–626, 2023.
- [25] Y. Sheng et al. FlexGen: High-throughput generative inference of large language models with a single GPU. *ICML*, pages 31094–31116, 2023.
- [26] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [27] NVIDIA. NVIDIA Blackwell Architecture Technical Brief. <https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/>. Accessed: January 15, 2026.
- [28] Z. Yao, X. Wu, C. Li, S. Youn, and Y. He. ZeroQuant-V2: Exploring post-training quantization in LLMs from comprehensive study to low rank compensation. *arXiv preprint arXiv:2303.08302*, 2023.

- [29] Z. Yuan et al. RPTQ: Reorder-based post-training quantization for large language models. *arXiv preprint arXiv:2304.01089*, 2023.
- [30] T. Dettmers et al. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. *arXiv preprint arXiv:2306.03078*, 2023.
- [31] W. Shao et al. OmniQuant: Omnidirectionally calibrated quantization for large language models. *ICLR*, 2024.
- [32] Y. Sun et al. FlatQuant: Flatness matters for LLM quantization. *arXiv preprint arXiv:2410.09426*, 2024.
- [33] V. Egiiazarian et al. Extreme compression of large language models via additive quantization. *arXiv preprint arXiv:2401.06118*, 2024.
- [34] A. Ghaffari et al. AdpQ: A zero-shot calibration free adaptive post training quantization method for LLMs. *arXiv preprint arXiv:2405.13358*, 2024.
- [35] E. Frantar et al. MARLIN: Mixed-precision auto-regressive parallel inference on large language models. *arXiv preprint arXiv:2408.11743*, 2024.
- [36] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [37] Z. Ye et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving. *arXiv preprint arXiv:2501.01005*, 2025.
- [38] H. Kang et al. TurboAttention: Efficient attention approximation for high throughputs LLMs. *arXiv preprint arXiv:2412.08585*, 2024.
- [39] R. Y. Aminabadi et al. DeepSpeed Inference: Enabling efficient inference of transformer models at unprecedented scale. *Proceedings of SC*, 2022.
- [40] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. *arXiv preprint arXiv:2211.17192*, 2022.
- [41] C. Chen et al. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [42] N. Bhendawade et al. Speculative streaming: Fast LLM inference without auxiliary models. *arXiv preprint arXiv:2402.11131*, 2024.
- [43] H. Xia et al. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. *Findings of EMNLP*, 2023.
- [44] P. Micikevicius et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.
- [45] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [46] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23(120):1–39, 2022.
- [47] Google. 2024 Environmental Report. <https://sustainability.google/reports/google-2024-environmental-report/>, 2024.

- [48] D. Patterson et al. The carbon footprint of machine learning training will plateau, then shrink. *Computer*, 55(7):18–28, 2022.
- [49] Google. 2024 Environmental Report. <https://sustainability.google/reports/google-2024-environmental-report/>. Accessed: January 20, 2026.
- [50] Google Cloud. Carbon free energy for Google Cloud regions. <https://cloud.google.com/sustainability/region-carbon>. Accessed: January 25, 2026.
- [51] Microsoft. 2025 Environmental Sustainability Report. <https://blogs.microsoft.com/on-the-issues/2025/05/29/2025-environmental-sustainability-report/>. Accessed: January 20, 2026.
- [52] Microsoft. Next-generation datacenters and efficiency tradeoffs. <https://www.microsoft.com/en-us/microsoft-cloud/blog/>. Accessed: January 20, 2026.
- [53] AWS. Sustainability: Data Centers. <https://aws.amazon.com/about-aws/sustainability/>. Accessed: January 22, 2026.
- [54] Amazon Sustainability. AWS PUE and WUE metrics. <https://sustainability.aboutamazon.com/products-services/aws-cloud>. Accessed: January 22, 2026.
- [55] International Energy Agency. Energy and AI. <https://www.iea.org/reports/energy-and-ai>. Accessed: January 18, 2026.
- [56] Uptime Institute. Global Data Center Survey 2024. <https://intelligence.uptimeinstitute.com/>. Accessed: January 18, 2026.
- [57] ISO/IEC 30134-2:2026. Data centres key performance indicators—Part 2: Power usage effectiveness (PUE). International Organization for Standardization, 2026.
- [58] US Environmental Protection Agency. Emissions & Generation Resource Integrated Database (eGRID) 2023. <https://www.epa.gov/egrid>. Accessed: January 25, 2026.
- [59] Ember. Global Electricity Review 2025. <https://ember-energy.org/insights/global-electricity-review-2025/>. Accessed: January 25, 2026.
- [60] electricityMaps. Real-time carbon intensity methodology and datasets. <https://www.electricitymaps.com/methodology>. Accessed: January 25, 2026.
- [61] National Renewable Energy Laboratory. Cambium 2024: Modeled hourly emissions metrics. <https://www.nrel.gov/analysis/cambium.html>. Accessed: January 25, 2026.
- [62] H. Wang et al. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, October 2023.
- [63] S. Ma et al. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, February 2024.
- [64] H. Wang et al. BitNet a4.8: 4-bit activations for 1-bit LLMs. *arXiv preprint arXiv:2411.04965*, November 2024.
- [65] H. Wang et al. BitNet v2: Native 4-bit activations with Hadamard transformation for 1-bit LLMs. *arXiv preprint arXiv:2504.18415*, April 2025.

- [66] S. Ma et al. BitNet b1.58 2B4T technical report. *arXiv preprint arXiv:2504.12285*, April 2025.
- [67] J. Zhang et al. Bitnet.cpp: Efficient edge inference for ternary LLMs. *arXiv preprint arXiv:2502.11880*, February 2025.
- [68] Microsoft. BitNet: Official inference framework for 1-bit LLMs. <https://github.com/microsoft/BitNet>. Accessed: January 28, 2026.
- [69] H. Peng et al. FP8-LM: Training FP8 large language models. *arXiv preprint arXiv:2310.18313*, October 2023.
- [70] B. Noune et al. Training and inference of large language models using 8-bit floating point. *arXiv preprint arXiv:2309.17224*, September 2023.
- [71] X. Wu et al. ZeroQuant-FP: A leap forward in LLMs post-training W4A8 quantization using floating-point formats. *arXiv preprint arXiv:2307.09782*, July 2023.
- [72] Y. Liu et al. FireQ: Fast INT4-FP8 kernel and RoPE-aware quantization for LLM inference acceleration. *arXiv preprint arXiv:2505.17305*, May 2025.
- [73] H. Badri and A. Shaji. HQQ: Half-quadratic quantization of large language models. <https://github.com/mobiusml/hqq>. Accessed: January 28, 2026.
- [74] Google Cloud. Cloud TPU v4 delivers strong MLPerf Training v2.0 results. <https://cloud.google.com/blog/products/ai-machine-learning/google-cloud-tpu-v4-mlperf-training-v2-results>. Accessed: January 20, 2026.
- [75] Google Cloud. Introducing Cloud TPU v5p and AI Hypercomputer. <https://cloud.google.com/blog/products/ai-machine-learning/introducing-cloud-tpu-v5p-and-ai-hypercomputer>. Accessed: January 20, 2026.
- [76] Google Cloud. TPU v5p specifications and system architecture. <https://cloud.google.com/tpu/docs/v5p>. Accessed: January 20, 2026.
- [77] AWS Neuron. Inferentia2 inference performance benchmarks. <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/general/benchmarks/inf2/inf2-performance.html>. Accessed: January 22, 2026.
- [78] GPUSStack. Inference performance lab: Ascend 910B NPU benchmarks. <https://docs.gpustack.ai/latest/tutorials/inference-performance-lab/>. Accessed: January 28, 2026.
- [79] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, December 2024.
- [80] H. Pan et al. Dense training, sparse inference: Rethinking training of mixture-of-experts language models. *arXiv preprint arXiv:2404.05567*, April 2024.
- [81] X. Liu et al. LSH-MoE: Communication-efficient MoE training via locality-sensitive hashing. *Advances in Neural Information Processing Systems*, 37, 2024.
- [82] Y. Huang et al. Elastic MoE: Unlocking the inference-time scalability of mixture-of-experts. *arXiv preprint arXiv:2509.16803*, September 2025.
- [83] Z. Wang et al. DualSparse-MoE: Coordinating tensor and neuron-level sparsity for efficient large language model deployment. *arXiv preprint arXiv:2508.16234*, August 2025.